

模块六: Web 应用架构

第二十二讲: 脚本服务化——将 Python 逻辑封装为 Web 服务

[解释]

本讲概览

本讲将学习如何用 FastAPI 框架将 Python 函数变为可通过浏览器访问的 Web 接口，然后用 HTML + JavaScript (Fetch) 构建前端页面来展示数据，完成一个完整的全栈天气应用。

一、搭建后端服务： FastAPI 入门

从调用别人的接口，到对外提供自己的接口

[解释]

本章定位

上一讲学习了如何调用别人的 API，本讲进入动手实操阶段——用 FastAPI 把 Python 函数发布为自己的 Web 接口。重点不是记住 FastAPI 的所有语法（具体代码可以让 AI 生成），而是理解“装饰器将 URL 路径绑定到处理函数”这个核心机制。

认识 FastAPI

Why FastAPI?

- **快 (Fast)**: 运行速度极快, 性能比肩 Go 和 Node.js。
- **易 (Easy)**: 代码简洁, 像写普通 Python 函数一样。
- **现代 (Modern)**: 自动生成交互式测试文档 (Swagger UI)。
- **AI 友好**: 因为语法极简, 大模型生成的代码质量非常高。

安装

```
pip install fastapi uvicorn
```

- **fastapi**: Web 框架本身, 负责路由和业务逻辑。
- **uvicorn**: ASGI 服务器, 负责接收和分发网络请求。



[解释]

为什么选 FastAPI

在 Python 世界里, 有很多做 Web 的工具 (比如 Django、Flask)。FastAPI 是最新的、最快的, 也是目前最适合与 AI 结合的。

它最大的优势是"所见即所得": 你只需要写 Python 函数, 它自动帮你变成网页接口, 还自动生成测试文档 (Swagger UI)。这极大降低了学习成本。

在 Python 世界里, Django 像一个"全包式酒店" (功能多但重), Flask 像一个"毛坯房" (灵活但要自己装修), 而 FastAPI 像一个"精装公寓"——开箱即用、功能齐全、结构整洁。

从 Hello World 开始 (1/2): 生成代码

指令（发给智能体）

"你好，我是 Python 初学者。请帮我用 FastAPI 编写一个最小 Web 服务代码，监听在 8000 端口，且允许局域网所有设备（0.0.0.0）访问。在根路由（/）返回一行提示：Hello World。"

AI 返回的参考代码

```
from fastapi import FastAPI
import uvicorn

app = FastAPI()                # 1. 创建应用实例

@app.get("/")                  # 2. 路由：绑定 URL 与处理函数
def root():
    return {"message": "Hello, World!"}

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000) # 3. 启动服务器
```

[解释]

核心指令解析

在使用 AI 生成代码时，指令的精准度决定了代码的可用性。

在这个基础案例中，我们明确指出了：

1. **技术栈**：FastAPI
 2. **网络配置**：监听 0.0.0.0 (允许非本机访问) 和端口 8000
 3. **业务逻辑**：在根路由 / 返回提示文字
- 养成这种包含业务约束细节的 Prompt 习惯，是进入 HOTL 开发模式的第一步。

从 Hello World 开始 (2/2): 代码解析

- `app = FastAPI()` : 创建一个 FastAPI 应用实例。
- `@app.get("/")` : 这叫**装饰器**。不用懂原理, 只要知道它把一个 URL 和一个 **函数** 绑在了一起。
- `return {...}` : 返回一个 Python 字典, FastAPI 自动把它变成 JSON。
- `host="0.0.0.0"` : **关键!** 表示监听所有网络接口, 允许局域网内的其他设备访问。如果是 `127.0.0.1`, 就只能在本机使用。

[解释]

装饰器 (Decorator)

代码里那个 `@` 符号不用去管复杂的定义。可以把它想象成便利贴或标签。

- `def root()` 是一个普通的 Python 函数。
- 加上 `@app.get("/")`, 就是给这个函数贴了个标签: "如果有人访问首页 (/), 就运行我"。FastAPI 通过装饰器将 URL 路径与处理函数绑定在一起。

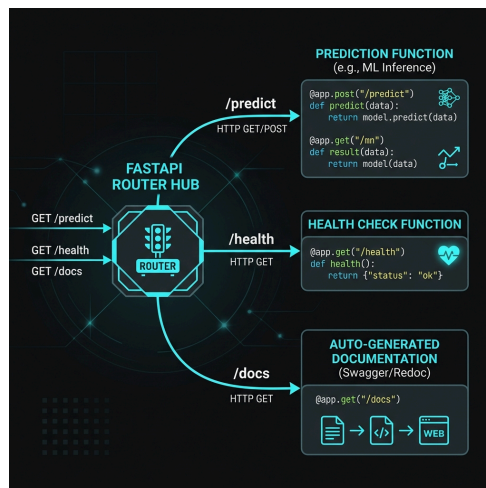
关键概念：路由与端点

路由 (Route)

- 什么是路由？： URL 路径与处理函数之间的映射关系。
- 代码对应： `@app.get("/...")` 里的那个字符串。
 - `/` → 根路径（首页）。
 - `/weather` → 天气数据接口。
 - `/forecast` → 预报查询接口。

端点 (Endpoint)

- 什么是端点？： 路由背后真正执行逻辑的 Python 函数。
- 代码对应： `def root():` 等指代的那段真实的 Python 代码。



简单记忆：路由 = URL 路径，端点 = 处理函数。用户访问正确的路径，对应的函数就会被执行。

[解释]

路由的工作机制

路由是 URL 路径与处理函数之间的映射关系。当用户访问 `/weather/Nanchang` 时，FastAPI 会在已注册的路由表中查找匹配项，找到后调用对应的处理函数。如果没有匹配的路由，自动返回 404 错误。

引入 SKILL 驱动 (1/2): HOTL 编程模式

复刻天气预报 API: 告别手写

- 刚才我们看到真实的 JSON 天气数据。现在我们要从体验者变成创造者。
- **HOTL 原则**: 遇到像外部请求、数据清洗这类复杂逻辑, 我们不手写代码。
- **标准操作**: 给大模型提供一份标准技术规范卡片:
`SKILL.md`。

指令 (发给大模型)

"@[SKILL.md] 附件是天气接口的技能规范文档 (基于 Open-Meteo 接口)。请帮我在原有的 FastAPI 代码里新增一个动态路由 `/weather/{city}`。当用户访问时, 请让程序在后台执行这个 SKILL (仅实现第1步: 提取坐标并拉取 JSON, 无需润色文本), 然后将其返回。"

技能挂载: 你负责设计业务流, AI 负责实现业务单元逻辑。

[解释]

什么是 SKILL.md

SKILL.md 是一份结构化的需求描述文档, 用于告诉 AI 编程助手需要实现什么功能、使用哪些接口、输出什么格式。

当业务逻辑较复杂时 (如需要发起外部 HTTP 请求、解析嵌套 JSON), 直接口头描述容易遗漏细节。将需求写成文档再交给 AI, 生成的代码质量会显著提高。

解析代码：路径参数的魔法

AI 生成的“胶水代码”

```
# AI 自动写好了近 50 行包含报错重试、经纬度解析
# 的底层函数 fetch_open_meteo(city)。
# 它最后会被精简封装在下面这段路由里：

@app.get("/weather/{city}")
def get_weather(city: str):
    # 调用 AI 生成的底层隐式技能函数
    weather_data = fetch_open_meteo(city)
    return weather_data
```

魔法解析：{city}

路径参数 (Path Parameter)

{city} 就像是 URL 里的完形填空。你在地址栏填入任意内容，FastAPI 会自动拦截这串字符，将其赋值给函数里的 city 变量。

- /weather/Nanchang → city = "Nanchang"
- /weather/Beijing → city = "Beijing"
- 一个接口函数，直接赋能全球天气查询！

[解释]

核心概念：技能挂载与动态路由

1. **为什么要写 SKILL.md**：它是一份外挂的知识库。在复杂的业务逻辑（如调用外部第三方天气接口）场景下，向 AI 提供标准的规范文档，能约束它的生成边界，极大地降低大模型幻觉，提高代码的首次编译成功率。
2. **什么是路径参数**：代码中的 {city} 是一种动态路由。它允许客户端通过 URL 直接传递变量给后端的 Python 函数。这意味着开发者只需维护这 1 个函数的代码，便能实时响应全球所有城市的天气请求，这是 API 架构高复用性的集中体现。

接口测试：Swagger UI

浏览器地址栏的局限

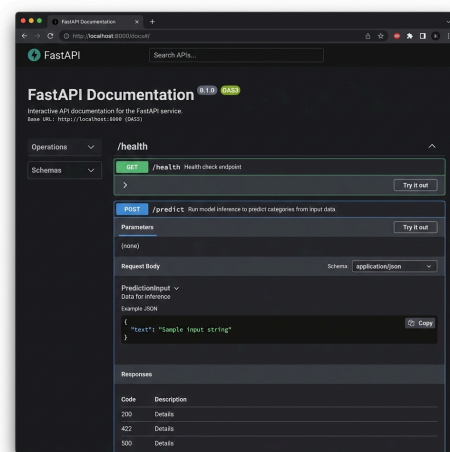
浏览器地址栏只能做最简单的 GET 查询。如果想做更专业的测试（比如 POST 提交数据），地址栏就不够用了。

FastAPI 的自动文档：/docs

FastAPI 自动为你生成了交互式测试界面。

操作：

1. 访问 `http://127.0.0.1:8000/docs`
2. 展开任意接口，点击 **Try it out**
3. 填写参数，点击 **Execute**



特别说明

- **Swagger UI** 是给开发者用的接口测试工具。
- 接下来要做的 **网页界面** 才是给最终用户使用的。

[解释]

自动生成的说明书

以前写完代码，还得专门写一份 Word 文档告诉别人怎么用你的接口。FastAPI 最厉害的地方在于：代码写完，使用说明书（API 文档）自动就生成好了。

Swagger UI 读取 FastAPI 根据代码中的类型注解和路由定义自动生成的 OpenAPI 规范文件，并将其渲染为可交互的测试界面。

二、从开发测试到用户界面

Swagger 是开发者的测试工具，用户需要的是可操作的界面

[解释]

为什么还需要前端

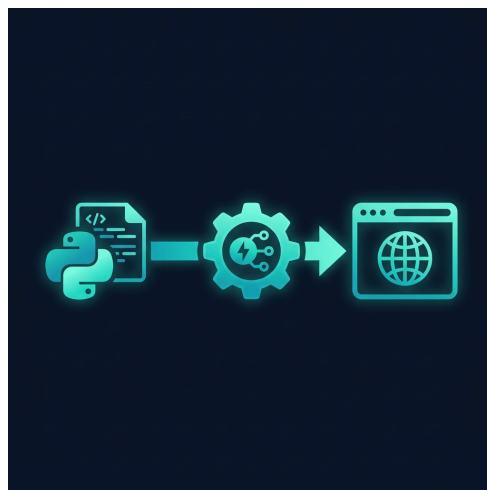
后端 API 只负责返回 JSON 数据，普通用户无法直接使用。前端（HTML + JavaScript）负责将数据渲染为可操作的界面。后端提供数据能力，前端提供交互体验，二者协作构成完整的 Web 应用。

前端交互：本环节核心目标

1. **前后端分离**：理解现代 Web 开发中前端和后端各自的职责。
2. **实现前端交互**：掌握 JavaScript 的 `fetch` 技术，让网页能向后端请求数据。
3. **上下文工程**：学会用 `PROJECT.md` 驱动 AI 生成高质量的全栈代码。

前端三层分工

- **HTML/CSS**：页面的静态结构与样式。
- **JavaScript (Fetch)**：负责动态获取和更新数据。
- **API**：前后端之间的数据接口。



[解释]

什么是"交互"

从"看内容"到"用工具"的跨越。以前的静态网页像是布告栏，用户只能被动阅读。现在的 Web 应用像是服务柜台，用户可以操作（点击、输入），并得到即时的反馈。

本节的核心就是掌握浏览器如何向服务器发起数据请求，并将返回结果更新到页面上。

三、前端如何获取后端数据：Fetch

让网页向后端发起请求的核心指令

[解释]

为什么选 Fetch

Fetch 是现代浏览器的标准 API，不需要安装任何额外的库。相比老式的 XMLHttpRequest，Fetch 语法更简洁、更接近自然语言的表达方式，非常适合初学者理解。

Fetch 三步曲 (1/2): 语法流程

Fetch API

浏览器内置的网络请求接口。

用法（人话版）：

1. 去哪里： `fetch('http://.../weather')`
2. 拿什么： `.then(拿到数据)`
3. 做什么： `.then(显示在屏幕上)`

不用死记语法：你只需要理解这个三步流程。具体代码让 AI 来写。

[解释]

Fetch 的本质

Fetch 就像是前端派出的“信使”。如果你让 AI 帮你写前端页面获取数据的代码，AI 必然会用到 Fetch。作为全栈设计的指挥官，你不必去背那些括号、箭头、异步处理等底层语法细节。你只要在需求里能够明确明确下达这三个核心动作：

1. 调用哪个接口（把信送给谁）
2. 收取什么格式的结果（信件带回的语言，通常为 JSON）
3. 怎么排布到网页里（拿到信后贴在哪里）

Fetch 三步曲 (2/2): 代码拆解

```
// JavaScript 代码片段 — 读的比写的更重要

// 1. 向后端发起请求
fetch('http://127.0.0.1:8000/weather')
  // 2. 等待回话, 把回话翻译成 JSON
  .then(response => response.json())
  // 3. 拿到数据, 更新屏幕
  .then(data => {
    // 把返回的数据填入网页
    document.getElementById('weather-display')
      .innerText = JSON.stringify(data);
  });
```

翻译：浏览器先去 `/weather` 请求气象数据 → 收到后把内容翻译成 JSON → 最后把翻译好的结果写到页面上 id 叫 `weather-display` 的位置。

[解释]

为什么要有 `.then` (然后)

网络请求需要时间，浏览器不会傻等着数据返回。`.then()` 的作用是告诉浏览器：“你先继续执行其他代码，等数据回来之后再执行 `.then()` 里面的逻辑。”这种机制叫做异步 (Asynchronous)，它保证了网页在等待数据时不会卡死。

目前只需记住 `.then()` 的执行顺序即可：先发请求 → 等数据到达 → 执行后续处理。

四、实战联调：全栈天气看板 MVP

敏捷工程哲学：先调通 (Make it Work)，再精修 (Make it Beautiful)

[解释]

先调通、再美化

在实际开发中，常见的错误是功能还没跑通就开始纠结界面样式。本节采用分步策略：第一步只验证前后端数据能否正确流转（不加任何 CSS），第二步确认无误后再美化界面。这种做法同样适用于给 AI 下指令——拆分为多个小步骤，比一次性提出复杂需求更容易得到正确结果。

第一站：先调通管道 (1/2)：下达指令

指令（发给 AI 助手）

"@[SKILL.md] 结合我们刚才基于这个文档生成的 FastAPI 天气接口 (/weather/{city})。

请帮我写一个最简单的 `index.html`：

1. 页面上只有一个文本打字框和一个搜索按钮。
2. 点击搜索后，使用 JavaScript 的 `fetch` 指令访问本机的气象接口。
3. 拿到天气数据后，不要加任何 CSS 修饰，将其作为普通文本印在网页底部即可。"

为什么这么朴素？

- **唯一的目的：**验证网页中的 JavaScript 能否成功调用本机运行的 Python 后端接口。

[解释]

工程法则：分离关注点

在构建 CS（前后端）架构的验证初期，最大的大忌就是“逻辑与视觉表现混杂”。

“先通管道”的意义在于：通过一个最极简的、甚至丑陋的“输入框+获取原生文本”的 HTML 页面，我们可以立即验证 CORS 跨域瓶颈是否解决、API 端口通信是否正常。

在这阶段如果有 Bug，100% 属于数据通信层的缺陷。等这个骨架验证通过后，再让 AI 糊上优美的 CSS 外观。这样如果视觉崩溃了，你能清晰地知道该修 CSS 而不是去怀疑数据链路，大幅降低了全栈排错的难度。

第一站：先调通管道 (2/2)：核心代码

生成的前端调度核心

```
// 当在框里输入南昌并点击搜索时触发
function searchWeather() {
  const city = document.getElementById("cityInput").value;
  // 向本机后端发起请求
  fetch("http://127.0.0.1:8000/weather/" + city)
    .then(res => res.json())
    .then(data => {
      // 将获取到的数据直接写入页面指定区域
      document.getElementById("result").innerText = JSON.stringify(data);
    })
    .catch(err => alert("出错了! " + err));
}
```

[解释]

为什么第一版不加样式

第一版页面故意不使用任何 CSS，目的是排除界面层的干扰。如果数据没有正确显示，可以确定问题出在 Fetch 请求或后端接口上，而不是 CSS 选择器写错了。这是一种常用的调试策略：先用最简方式确认核心逻辑正确，再逐步增加复杂度。

常见问题：跨域阻断 (CORS)

你一定会遇到的报错

当你在网页中点击“搜索天气”时，页面没有任何反应。按下 `F12` 打开控制台，你会看到一行醒目的红色报错：

```
Access to fetch ... has been  
blocked by CORS policy
```

原理剖析

这是浏览器的同源安全策略在拦截你：

1. 你的网页叫 `index.html`（文件协议）。
2. 你的后端运行在 `127.0.0.1:8000`（HTTP 协议）。

两者协议不同，属于不同源。浏览器为防止恶意网页跨站窃取数据，默认禁止此类请求。

✗ Fetch Error: Failed to fetch.

▶ Access to fetch at `http://127.0.0.1:8000/weather/Nanchang` from origin `null` has been blocked by CORS policy: No Access-Control-Allow-Origin header is present on the requested resource.

[解释]

什么是同源策略

浏览器出于安全考虑，默认禁止网页向不同源（协议、域名或端口不同）的服务器发请求。这就是遇到 CORS 报错的原因：`index.html` 通过 `file` 协议打开，而后端运行在 `http://127.0.0.1:8000`，两者不同源。CORS（Cross-Origin Resource Sharing）是服务器主动声明“我允许哪些来源访问我”的机制。

跨域放行：配置 CORS 中间件

修复方案

问题出在服务端——我们的 FastAPI 没有声明允许跨域访问。

我们只需在 `main.py` 中添加 CORS 配置。

指令（发给 AI 助手）

"我刚才用 HTML 页面调用你的

`/weather/{city}` 时遇到了 CORS 跨域报错。

请帮我在 `main.py` 里加上 FastAPI 的 CORS 中间件配置（允许所有来源访问），然后再把完整的 `main.py` 给回我。"

补救后的代码（关注高亮部分）

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

# 配置跨域访问策略
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # 允许所有来源访问（仅用于开发环境）
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/weather/{city}")
def get_weather(city: str):
    ...
```

这回真通了：重新启动 `python main.py`，再试一次 HTML 的搜索。天气数据成功显示在页面上。前后端通信正式打通。

[解释]

什么是中间件

中间件（Middleware）是在请求到达业务逻辑之前自动执行的处理层。`CORSMiddleware` 会在每个响应中添加 `Access-Control-Allow-Origin` 等 HTTP 头，告诉浏览器允许跨域访问。`allow_origins=["*"]` 中的星号表示允许所有来源，仅适用于开发阶段。正式部署时应替换为具体的域名。

第三站：界面美化——感受 AI 生成前端的效率

指令（发给 AI 助手）

"前后端通信已经调通，数据流转正常。

现在请帮我美化界面。请按照现代前端设计风格：

1. 请用毛玻璃（Glassmorphism）风格重新设计 `index.html` 的界面。
2. 页面使用气象主题的渐变背景色。
3. 原本的 JS 数据获取逻辑不要变。但当数据回来时，请把返回数据中的 `temp`（温度）、`humidity`（湿度）、`desc`（天气描述）提取出来，填进网页里面漂亮的卡片排版里！"

美化后的效果



核心认知：你花一节课搭建的 Fetch 数据通道，才是应用的骨架。这层精美的 CSS 界面，只是 AI 几秒钟生成的外观。

[解释]

数据通道与界面的关系

对比朴素版和美化版可以发现：两个版本的 JavaScript Fetch 逻辑完全相同，区别仅在于 HTML 结构和 CSS 样式。

这说明前后端数据通道（Fetch 请求 → JSON 解析 → 页面渲染）是应用的核心骨架，界面样式是可以随时替换的表现层。在 AI 辅助开发中，样式代码的生成效率很高，开发者应将主要精力放在接口设计和数据流转的正确性上。

五、深度思考与范式 升华

从"写代码"到"指挥 AI 造产品"

[解释]

思考：刷新后数据消失

刷新页面后之前查询的天气数据消失了，这不是 Bug，而是 HTTP 协议的设计特性——无状态（Stateless）。服务器处理完每个请求后不会记住之前的交互。如果需要持久保存数据，就需要引入数据库等存储机制。

破坏性实验：刷新一下试试？

现象观察

请做一个实验：刷新浏览器页面。

- 之前查询的天气数据.....全部消失了。
- 服务器无法区分当前请求和之前的请求。

这是 Bug 吗？

不，这是 HTTP 协议的核心设计特征——无状态 (Stateless)。

类比

服务器就像一台自动售货机：你买完一瓶水走开，再回来买第二瓶，它并不认识你。每一次请求都是独立的。

设计权衡 (Trade-off)

	优点	缺点
无状态	服务器负载低，可轻松应对百万并发	无法维持"会话"，刷新即丢失

怎么解决？

真实的企业级应用会引入：

- 数据库：把订单持久化保存
- Session/Token：让服务器"记住"用户

💡 当前的"丢失"是 MVP 阶段的有意简化。理解了原因，你就能在需要时告诉 AI："请加上数据库持久化"。

[解释]

无状态协议 (Stateless Protocol)

HTTP 协议是无状态的：服务器处理完一个请求后，不会记住之前的任何信息。

- 优点：服务器不需要为每个用户维护状态，可以高效处理大量并发请求。
- 缺点：如果需要"连续对话"（如购物车、登录状态），应用层必须自己引入存储机制（如 Session、Cookie 或数据库）。

这解释了为什么刷新页面后之前查询的数据会消失——因为服务器根本不记得你之前查了什么。

AI 赋能的全栈开发范式 (1/2)

我们今天走过的完整流程

1. 🎯 定义 (PM): 梳理模糊需求 → 功能清单
2. 🏗️ 设计 (Architect): 定义 API 契约 → `PROJECT.md`
3. 🤖 生成 (AI Worker): AI 基于 `PROJECT.md` → 后端 + 前端
4. ✅ 验证 (QA): Swagger 测后端 → 浏览器测全栈

角色切换是核心能力

你不是在逐行写代码，而是在协调分工：

- 给 AI 当 PM → 整理需求
- 给 AI 当架构师 → 定接口
- 给 AI 当 QA → 验收测试

AI 赋能的全栈开发范式 (2/2)

为什么先学后端再做全栈？

虽然 AI 能一口气生成全栈代码，但如果不理解 API 和 JSON，你就无法做质量品控。

- 第 21 讲：理解 Web 架构与数据格式
- 第 22 讲：动手搭建全栈应用

核心公式

Coding is cheap, Architecture & Verification is expensive.

代码变得廉价（AI 秒生成），而架构设计与验收能力变得昂贵（需要人类判断）。

[解释]

AI 辅助开发中的人机分工

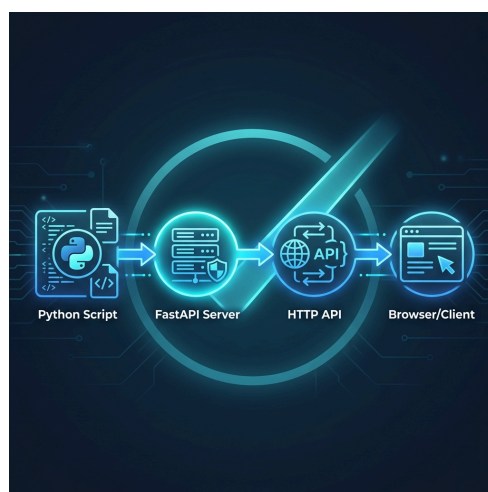
在本讲的实践中，大部分代码由 AI 生成，而我们负责的是：确定需求（做什么接口）、设计架构（前后端如何分工）、验证结果（接口返回是否正确、页面是否正常显示）。

AI 擅长快速生成代码，但架构设计和结果验证仍需要人来把控。这种“人负责决策和验收、AI 负责执行”的协作模式，是 HOTL（Human-On-The-Loop）的具体体现。

课程小结

今日关键收获

1. **Fetch 三步曲**：去哪里 → 拿什么 → 做什么——前后端通信的核心。
2. **PROJECT.md 上下文工程**：用结构化文档驱动 AI 生成一致的全栈代码。
3. **全栈闭环**：后端 (FastAPI) + 前端 (HTML/JS) + 通信 (Fetch/JSON) = 完整 Web 应用。
4. **工程素养**：CORS 安全机制 + 接口对齐审查。
5. **开发范式**：PM → Architect → AI Worker → QA 的螺旋迭代。



下节预告

我们已经会从零开始做全栈应用了。但你之前写过那么多 Python 脚本——数据分析、文本处理、绘图.....它们还只能在本地终端运行。

下节课，下节课，我们会把它们封装成 Web 接口，加上前端界面，让其他人也能通过浏览器使用。

[解释]

全栈开发的三个核心概念

本讲涉及的技术细节较多，但核心就是三件事：

1. **解耦**：前端和后端各自独立运行，通过 API 通信，互不干扰。
2. **契约**：前后端提前约定好接口格式（URL、参数、返回值），确保双方数据能对接。
3. **闭环**：从确定需求 → 生成代码 → 测试验证 → 上线运行，构成一个完整的开发流程。